

Airdrop New *Dappos*

HALBORN

Airdrop New - Dappos

Prepared by:  HALBORN

Last Updated 12/11/2025

Date of Engagement: December 5th, 2025 - December 5th, 2025

Summary

100%  OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
7	0	0	0	3	4

TABLE OF CONTENTS

- 1. Introduction
- 2. Assessment summary
- 3. Test approach and methodology
- 4. Risk methodology
- 5. Scope
- 6. Assessment summary & findings overview
- 7. Findings & Tech Details
 - 7.1 Centralization risks
 - 7.2 Single step ownership transfer process
 - 7.3 Use of non-compliant tokens may break contract functionality
 - 7.4 Checkeligibility() does not consider pause state or contract balance
 - 7.5 Use of revert strings instead of custom errors
 - 7.6 Floating pragma
 - 7.7 Typos

1. INTRODUCTION

Dappos engaged Halborn to perform a security assessment of their smart contracts on December 4th, 2025. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The Dappos codebase in scope consists of a smart contract implementing an ERC20 airdrop mechanism using a Merkle tree for eligibility verification, with pause controls, claim deadlines, and post deadline token recovery for the contract owner.

2. ASSESSMENT SUMMARY

Halborn was allocated 1 day for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, Halborn identified several areas for improvement to reduce the likelihood and impact of security risks, which were mostly addressed by the Dappos team. The main recommendations were:

- Consider implementing a role-based system for access control, with a multisignature wallet setup, establishing community driven governance, and/or integrating time locks.
- Consider using OpenZeppelin's Ownable2Step contract over the Ownable implementation.
- Replace all direct ERC20 token operations with OpenZeppelin's SafeERC20 library.

3. TEST APPROACH AND METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (Foundry).
- Fork testing against main networks (Foundry).
- Static security analysis of scoped contracts, and imported functions (Slither).

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY
(a) Repository: AirdropNew (b) Assessed Commit ID: 5adb03b (c) Items in scope: contracts/Airdrop.sol Out-of-Scope: Third party dependencies and economic attacks.
REMEDIATION COMMIT ID:
7b38fdf 4023453 2878dd8 88a0efc 278bd41
Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW
0	0	0	3
INFORMATIONAL			
4			

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
CENTRALIZATION RISKS	LOW	SOLVED - 12/08/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
SINGLE STEP OWNERSHIP TRANSFER PROCESS	LOW	SOLVED - 12/08/2025
USE OF NON-COMPLIANT TOKENS MAY BREAK CONTRACT FUNCTIONALITY	LOW	SOLVED - 12/08/2025
CHECKELIGIBILITY() DOES NOT CONSIDER PAUSE STATE OR CONTRACT BALANCE	INFORMATIONAL	SOLVED - 12/07/2025
USE OF REVERT STRINGS INSTEAD OF CUSTOM ERRORS	INFORMATIONAL	ACKNOWLEDGED - 12/07/2025
FLOATING PRAGMA	INFORMATIONAL	SOLVED - 12/07/2025
TYPOS	INFORMATIONAL	SOLVED - 12/07/2025

7. FINDINGS & TECH DETAILS

7.1 CENTRALIZATION RISKS

// LOW

Description

The `Airdrop` contract grants the owner significant control over critical protocol parameters and functionality. The owner's privileges include:

- **Setting the Merkle root at any time** via `setMerkleRoot()`, which can invalidate pending claims in the mempool or change who is eligible mid-airdrop.
- **Modifying the claim deadline arbitrarily** via `setClaimEndTime()`, which can shorten the claim window without notice, cutting off legitimate claimants.
- **Withdrawing all tokens** via `withdraw()` once `claimEndTime` passes or `merkleRoot` is unset, which combined with `setClaimEndTime` allows immediate fund extraction.
- **Pausing and unpausing claims** via `pause()` / `unpause()`, which can freeze user claims indefinitely while owner actions remain unrestricted.
- **Performing state changes while paused**, meaning the owner can modify the Merkle root, deadline, or withdraw funds even while claims are blocked.

These privileges create a centralization risk that could lead to owner privileges being abused if the owner becomes malicious or their credentials are compromised,

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:C (2.5)

Recommendation

Several remedial strategies can be employed, including but not limited to: implementing a role-based system for access control, transitioning control to a multi-signature wallet setup for critical functions, establishing community-driven governance for decision-making on fund management, and/or integrating time locks.

Remediation Comment

SOLVED: The **Dappos team** solved this finding by clarifying that:

Our plan is to use a Safe multisig wallet to control the owner privileges.

7.2 SINGLE STEP OWNERSHIP TRANSFER PROCESS

// LOW

Description

The `Airdrop` contract inherits the `Ownable` contract implementation from OpenZeppelin's library and is used to restrict access to certain functions to the contract owner. The `Ownable` pattern allows the contract owner to transfer ownership to another address using the `transferOwnership()` function. However, the `transferOwnership()` function does not include a two-step process to transfer ownership.

Regarding this, it is crucial that the address to which ownership is transferred is verified to be active and willing to assume ownership responsibilities. Otherwise, the contract could be locked in a situation where it is no longer possible to make administrative changes to it.

Additionally, the `renounceOwnership()` function allows renouncing to the owner permission. Renouncing ownership before transferring it would result in the contract having no owner, eliminating the ability to call privileged functions.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Consider using OpenZeppelin's `Ownable2Step` contract over the `Ownable` implementation. `Ownable2Step` provides a two-step ownership transfer process, which adds an extra layer of security to prevent accidental ownership transfers.

Additionally, it is recommended that the owner cannot call the `renounceOwnership()` function to avoid losing ownership of the contract.

Remediation Comment

SOLVED: The **Dappos team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/DappOSDao/AirdropNew/commit/7b38fdff144e8768d965a334bd448c9382072393>

7.3 USE OF NON-COMPLIANT TOKENS MAY BREAK CONTRACT FUNCTIONALITY

// LOW

Description

In the `Airdrop` contract, there are instances where direct `ERC20` operations are performed without using safety wrappers.

In `claim()`:

 Copy Code

```
78 | require(token.transfer(msg.sender, amount), "Transfer failed");
```

In `withdraw()`:

 Copy Code

```
158 | require(token.transfer(owner(), amount), "Transfer failed");
```

This functionality assumes that all `ERC20` token transfers comply with the `IERC20` standard, which is not always the case. For example, USDT's `transfer` and `transferFrom` functions do not return a boolean as required by the standard. Other problematic implementations include tokens that revert on zero-amount transfers and fee-on-transfer tokens that result in the received amount being less than the expected amount.

If non-standard `ERC20` tokens are used with these contracts, the operations may fail unexpectedly or silently, potentially breaking core protocol functionality and endangering user funds.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N (2.0)

Recommendation

Replace all direct ERC20 token operations with OpenZeppelin's `SafeERC20` library. This library provides wrapper functions that properly handle the edge cases of non-standard ERC20 implementations.

Remediation Comment

SOLVED: The **Dappos team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/DappOSDao/AirdropNew/commit/4023453b1768c5f623e59f151c44d6c2670664c1>

7.4 CHECKELIGIBILITY() DOES NOT CONSIDER PAUSE STATE OR CONTRACT BALANCE

// INFORMATIONAL

Description

The `checkEligibility()` function is intended to help users and frontend applications determine whether a wallet can successfully claim tokens. However, it does not account for the contract's pause state or its token balance when computing the `canClaim` value.

As a result, `canClaim` may return `true` even when the contract is paused or does not have enough tokens to fulfill the claim, which would cause the actual `claim()` call to revert. This can mislead users and integrations into believing a claim will succeed when it will actually fail.

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:L/D:N/Y:N (0.6)

Recommendation

Consider updating the `canClaim` calculation in `checkEligibility()` to include checks for the contract's pause state and token balance.

Remediation Comment

SOLVED: The **Dappos team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/DappOSDao/AirdropNew/commit/2878dd8d96ad6ee8f643025919d07d80e449ca43>

7.5 USE OF REVERT STRINGS INSTEAD OF CUSTOM ERRORS

// INFORMATIONAL

Description

Throughout the files in scope, there are several instances where revert strings are used over custom errors.

In Solidity development, replacing hard-coded revert message strings with the `Error()` syntax is an optimization strategy that can significantly reduce gas costs. Hard-coded strings, stored on the blockchain, increase the size and cost of deploying and executing contracts.

The `Error()` syntax allows for the definition of reusable, parameterized custom errors, leading to a more efficient use of storage and reduced gas consumption. This approach not only optimizes gas usage during deployment and interaction with the contract but also enhances code maintainability and readability by providing clearer, context-specific error information.

BVSS

AO:A/AC:H/AX:H/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (0.3)

Recommendation

Consider replacing all revert strings with custom errors. For example:

```
error ConditionNotMet();  
  
if (!condition) revert ConditionNotMet();
```

 Copy Code

or starting from Solidity `0.8.27` :

```
require(condition, ConditionNotMet());
```

 Copy Code

For more references, [see here](#) and [here](#).

Remediation Comment

ACKNOWLEDGED: The Dappos team made a business decision to acknowledge this finding and not alter the contracts.

7.6 FLOATING PRAGMA

// INFORMATIONAL

Description

The contracts in scope currently use floating pragma version `^0.8.28`, which means that the code can be compiled by any compiler version that is greater than `0.8.28`, and less than `0.9.0`. An outdated pragma version might introduce bugs that affect the contract system negatively.

Additionally, from Solidity versions `0.8.20` through `0.8.24`, the default target EVM version is set to `Shanghai`, which results in the generation of bytecode that includes `PUSH0` opcodes. Starting with version `0.8.25`, the default EVM version shifts to `Cancun`, introducing new opcodes for transient storage, `TSTORE` and `TLOAD`.

If deploying to networks other than Ethereum mainnet, the selected EVM version should be supported. Otherwise, unsupported opcodes may cause deployment failures or transaction issues.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Lock the pragma version to the same version used during development and testing (for example: `pragma solidity 0.8.28;`), and make sure to specify the target EVM version when using Solidity versions from `0.8.20` and above if deploying to chains that may not support newly introduced opcodes.

Remediation Comment

SOLVED: The **Dappos team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/DappOSDao/AirdropNew/commit/88a0efc97a7cd049382014802a39d038074868a9>

7.7 TYPOS

// INFORMATIONAL

Description

In the `Airdrop` contract, there are typos where the word `Set` is misspelled as `Setted` in the `MerkleRootSetted` and the `ClaimEndTimeSetted` events.

While these typos do not affect the functionality of the code, they can make the codebase harder to read and understand.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Fix all typos to improve the readability of the codebase.

Remediation Comment

SOLVED: The **Dappos team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/DappOSDao/AirdropNew/commit/278bd41a7d11999516b9eca778f2df8143b0700c>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.